

Byte-Pair Encoding (BPE) Tokenization

Byte-Pair Encoding (BPE) was first created as an algorithm for compressing text. Later, OpenAI used it for tokenization in GPT model pretraining. Now BPE is used in many Transformer models, such as:

GPT
GPT-2
RoBERTa
BART
DeBERTa

Simply put, BPE breaks large words into smaller parts and makes tokens from them, so that the model can learn words more effectively.

What Does BPE Actually Do?

Suppose, instead of keeping a whole word, a model keeps smaller parts of the word.

For example:

hugs → hug + s
playing → play + ing

In this way, the model can also understand new words, because it already learns the parts of words in advance.

1) Training Algorithm — How Does BPE Learn?

BPE training starts by taking all the unique words from a corpus (meaning a large text dataset).

Suppose our corpus has these 5 words:

"hug", "pug", "pun", "bun", "hugs"

Now, the letters used to write these words will be used to build the first vocabulary.

That will be:

["b", "g", "h", "n", "p", "s", "u"]

This means that at the beginning, the vocabulary will contain only single-character tokens.

What Happens in Reality?

In real cases, the base vocabulary usually contains:

all ASCII characters
many Unicode characters as well

If, during tokenization, a character appears that was not in the training corpus, then it becomes an unknown token ([UNK]).

That is why many NLP models cannot handle emojis or unusual characters very well.

GPT-2 / RoBERTa's Clever Solution

GPT-2 and RoBERTa use a clever trick.

Instead of looking at words as characters, they look at words as bytes.

Because of this:

the vocabulary size stays small (a total of 256 bytes)
but almost every character can still be represented
the problem of becoming an unknown token is reduced

This is called:

Byte-level BPE

2) What Is a Merge Rule?

After getting the initial vocabulary, BPE learns merge rules to create new tokens.

A merge rule means:

among the tokens in the vocabulary, the two tokens that appear next to each other most frequently are joined together to create a new token.

At first, merging creates 2-character tokens, later 3-character tokens, and then even larger subwords.

3) Let's Understand with an Example

Suppose the word frequencies in the corpus are:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

This means:

"hug" appears 10 times

"pug" appears 5 times

"pun" appears 12 times

"bun" appears 4 times

"hugs" appears 5 times

First Step: Splitting the Words into Characters

At the beginning, each word will be split into characters:

("h" "u" "g", 10)

("p" "u" "g", 5)

("p" "u" "n", 12)

("b" "u" "n", 4)

("h" "u" "g" "s", 5)

Second Step: Finding the Most Frequently Used Pair

Now let's see which two consecutive tokens appear most often.

Pair: ("h", "u")

It appears in:

hug → 10 times
hugs → 5 times
Total = 15 times

Pair: ("u", "g")
It appears in:

hug → 10 times
pug → 5 times
hugs → 5 times
Total = 20 times

This is the most frequent pair.

So the first merge rule will be:

("u", "g") -> "ug"

This means u and g combine to form the new token ug.

Now the vocabulary will be:

["b", "g", "h", "n", "p", "s", "u", "ug"]

And the corpus will be:

("h" "ug", 10)
("p" "ug", 5)
("p" "u" "n", 12)
("b" "u" "n", 4)
("h" "ug" "s", 5)

Third Step: Again Finding the Most Frequent Pair

Now let's look at the pairs again.

Pair: ("u", "n")
It appears in:

pun → 12 times
bun → 4 times
Total = 16 times

Pair: ("h", "ug")

It appears in:

hug → 10 times

hugs → 5 times

Total = 15 times

So this time the most frequent one is:

("u", "n") -> "un"

Now the vocabulary will be:

["b", "g", "h", "n", "p", "s", "u", "ug", "un"]

And the corpus will be:

("h" "ug", 10)

("p" "ug", 5)

("p" "un", 12)

("b" "un", 4)

("h" "ug" "s", 5)

Fourth Step: Merge Again

Now the most frequent pair is:

("h", "ug")

Because it appears in:

hug → 10 times

hugs → 5 times

Total = 15 times

So the merge rule will be:

("h", "ug") -> "hug"

Now the vocabulary will be:

["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]

And the corpus will be:

("hug", 10)
("p" "ug", 5)
("p" "un", 12)
("b" "un", 4)
("hug" "s", 5)

What Happens Next?

In this way, merging will continue until the desired vocabulary size is reached.

What could be the next merge rule here?

Most likely it will be:

("p", "un") -> "pun"

Because the pair ("p","un") appears a total of 12 times.

4) Tokenization Algorithm — How Are New Words Tokenized?

After training, when a new input comes, BPE follows these steps:

Normalization

bringing the text into a standard form

Pre-tokenization

doing some basic splitting beforehand

splitting the word into characters

applying the merge rules learned during training one by one

The learned merge rules were:

("u", "g") -> "ug"
("u", "n") -> "un"
("h", "ug") -> "hug"

Example 1: "bug"

At first it will be split as:

"b" "u" "g"

Then:

("u", "g") → ug

So the final token will be:

["b", "ug"]

Example 2: "mug"

Initial split:

"m" "u" "g"

But m was not in the base vocabulary.

So m will become [UNK]

And u and g will merge into ug.

Final:

["[UNK]", "ug"]

Example 3: "thug"

Initial split:

"t" "h" "u" "g"

Here t is not in the vocabulary, so:

["[UNK]" "h" "u" "g"

Then:

("u", "g") → ug

("h", "ug") → hug

The final token will be:

["[UNK]", "hug"]

Why Is BPE Needed?

BPE is needed because:

- it is not possible to keep all words in the vocabulary
- new words appear often
- subword-based tokenization makes the model more flexible
- rare words or unknown words become easier to handle
- memory and efficiency improve